Vivasoft Client Interview Prep (Bkash)

rasel.hasan@vivasoftltd.com

Web security

How will you secure your frontend app in terms of web security?

1. Using https
2. Using CSP (Content security policy, ki type er content kon source theke browser load korbe ta bole deya jay in http header, Mitigates XSS by preventing inline scripts, disallowing untrusted sources)
3. Use http only cookies
4. Sanitize all untrusted html (using dompurify)
5. Validate all inputs
6. Avoid inline js or eval
7. Disable browser auto complete for sensitive data
8. Restrict iframes and 3rd party scripts
9. Use secure routing with role check
10. Keep dependencies updated
11. Remove unused codes and debug statements
12. Avoid exposing secrets in the front end (avoid exposing user IDs in URLs.)
13. Use Subresource Integrity (SRI) for CDN assets. (Add the integrity attribute: Include the generated hash in the integrity attribute of the <script> or <link> tag that references the CDN asset.)

Web accessibility

How to make a website accessible?

Semantic html

1. Prefer button over clickable divs
2. Use semantic html tags, nav, main, header, footer, section, article for page structure, maintain heading hierarchy(h1 - h6), this will help screen readers & keyboard navigation automatically.

Keyboard accessibility

1. All interactive elements must be reachable via tab, enter, space and arrow keys.
2. Manage focus properly after modal/dialog actions.

Proper Area Usage/Definitions

1. Only use ARIA when semantic html cannot express something, like-
    a. aria-role for custom build elements, like role='dialog' for modals

    b. area-expanded, aria-controls for collapsible items
    c. Aria-label(where text is not shown) or aria-labelledby for unnamed icons/buttons.

Color Contrast ratio should be maintained, use WCAG 2.1 standards

Text alternatives- all images must have alt text.

Form accessibility

1. Every input must have a label or aria label
2. Use id + for linking
3. Provide clear error messages, not just red borders.

Responsive and zoom friendly

1. Layout should work at 200% zoom.

Content types

1. Audio/video content must be provided with subtitles.

How do you test accessibility manually?

1. Use Tab key to navigate.
2. Use a screen reader (VoiceOver, NVDA).
3. Check color contrast tools.
4. Run Lighthouse or axe-core.

WCAG-POUR Principle?

1. Perceivable (Shune ba dekhe use kora jay)
    a. Image thakle text alternative thaktei hobe.
    b. Color diye meaning deya jabe na, let say red means error, error-message e provide korte hobe.
    c. 200% zoom e page break kora jabe na.
    d. Horizontal scroll avoid korte hobe.
2. Operable (Keyboard diye use kora jay)
    a. Sob interactive element keyboard reachable hote hobe
    b. Focus dekha jabe
    c. No seizure risk: Mrigi rogir jeno kono issue na hoy, animation flash <= 3 times/sec.
3. Understandable (User bojhte pare)
    a. Clear label and instructions, input e <label> provide kora, placeholder ar label ek jinish na
    b. Error clearly bola, like ‘Email format is wrong’ instead of ‘Invalid input’
    c. Consistent behavior, same action -> same result ensure kora, navbar position change kora jabe na.
4. Robust (Future proof & assistive tech friendly)
    a. Semantic html use kora, like <button> for click, <a> for link, <select> for dropdown
    b. Aria rules (Native first, aria last)

WCAG is based on **POUR** principles. We need to ensure content is **perceivable** with proper text alternatives and contrast, **operable** via keyboard, **understandable** with clear labels and errors, and **robust** using semantic HTML and minimal ARIA, targeting WCAG 2.1 AA compliance.

React

Difference between react component and element?

A react element is an immutable object that represents only a portion of UI in a specific point of time, if UI needs to be updated, the element needs to be re-created.
A component is a reusable function that returns a react element and contains state and logic.

Rules of hooks

1. **Only call hooks at the top level**: Must be called at the top level of react functional components or custom hooks before any return statements. This ensures the order of hook calls in each render.
2. Naming convention: need to use '**use'** prefix with camelCase naming of the hook.

What is useMemo, its use cases?

useMemo is a react hook used to memoize expensive computed values so they are recalculated only when dependencies change, improving performance.
UseCases: loop, filters, sorting.

```
const filteredUsers = useMemo(() => {
  return users.filter(user => user.isActive);
}, [users]);
```

What is useCallback, why is it needed?

useCallback memoizes a function reference so it doesn't get recreated on every render, helping prevent unnecessary re-renders and effect executions.

```
const memoizedFn = useCallback(() => {
      doSomething(a, b);
}, [a, b]);
```

Component lifecycle methods in frontend frameworks

1. Mounting: The component is being created and inserted into the dom for the first time.

2. Updating: The component is re-rendering based on the props or state change.
3. Unmounting: The component is being removed from the dom.

Differences between useEffect and useLayoutEffect

1. useEffect runs asynchronously after the browser has painted the screen. Non blocking side effects, like data fetching.
2. useLayoutEffect runs synchronously before the browser paints the screen, mostly used to resolve visual flickering issues, to achieve precision level accuracy in dom measurements.

What is HOC (Higher order component)?

Higher order component is a react function that receives a react component as argument and returns its improved version with additional properties/behaviors. It is used to reuse component logic.

Render props pattern

```
function MouseTracker({ render }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });

  return (
    <div onMouseMove={e => setPos({ x: e.clientX, y: e.clientY })}>
      {render(pos)}
    </div>
  );
}

function App() {
  return (
    <MouseTracker
      render={(pos) => <h2>Mouse position: {pos.x}, {pos.y}</h2>}
    />
  );
}
```

NextAuth

Authentication flow:

Four callbacks by nextauth: signIn -> jwt -> session -> redirect

signIn: responsible for checking if user will be able to login or not, runs exact before being authentication successful by credentials or social login(oauth)
Jwt: responsible for creating / updating jwt token with custom properties
Session: responsible for creating the session object that will be accessible from client side, like appending user info, role, permissions from jwt token.
Redirect: responsible for handling redirects, like after authentication redirection to dashboard, handles external redirects too from a security perspective, and adds a custom logic layer before approving external redirects.

Javascript

What is closure, real life use cases of closure?

Closures allow a function to access variables from its outer scope even after that scope has returned. In frontend development, closures are heavily used in debouncing, throttling, event handlers, memoization, private state, and module patterns. They enable encapsulation and stateful function behavior without classes or global variables.

```
function debounce(fn, delay) {
    let timer;
    return function(...args) {
        clearTimeout(timer);
        timer = setTimeout(() => fn.apply(this, args), delay);
    }
}

const createStore = (initialValue) => {
    let store = initialValue;

    return ({
        setStore: (value) => {
            store = value;
        },
        getStore: () => {
            return store;
        }
    });
```

```
}

const store = createStore('init');
store.getStore(); // init

store.setStore('changed_value');
store.getStore(); // changed_value

function memoize(fn) {
    const cache = {};
    return function(n) {
        if(cache[n]) return cache[n];
        return(cache[n] = fn(n));
    }
```

This keyword

This keyword refers to the **execution context** of the current function. Its value is not set when the function is defined, but rather when the function is called.

The value of this is dynamically determined by how a function is invoked.

1. Global Context(default binding)
    a. When a function is called in the global scope(outside of any explicit object or class), **this** refers to the **global object**(e.g., **window** in a browser, **global** in nodejs), in strict mode if **this** is not set by the execution context it remains **undefined**.
2. Method Invocation (Implicit binding)
   When a function is called as a method of an object, **this** refers to the object that the method belongs to.
   ```
   const user = {
     name: 'Alice',
     greet: function() {
       // 'this' refers to the 'user' object
       console.log(`Hello, my name is ${this.name}`);
     }
   };
   user.greet(); // Output: Hello, my name is Alice
   ```
3. Constructor Invocation (new binding)
   When a function is used as a constructor (invoked with the **new** keyword), a new object is created, and **this** is bound to that newly created object.
   ```
   function Car(make) {
     // 'this' refers to the new object being created
     this.make = make;
   ```

```
}
const myCar = new Car('Toyota');
```

4. Explicit Binding (using call, apply and bind)
   We can manually set the value of this for a function call using call, apply and bind.
   1. **call(thisArg, arg1, arg2, …)**
      a. Immediately executes the function
      b. Set the this value to thisArg
      c. Accepts arguments individually after the thisArg.
      d. Usecase: when the arguments are known individually.
   2. **apply(thisArg, [argsArray])**
      a. Immediately executes the function
      b. Sets the this value to thisArg
      c. Accepts arguments as an array or array-like object.
      d. Usecase: When arguments are grouped in an array
   3. **bind(thisArg, arg1, arg2, …)**
      a. Returns a new function without executing immediately.
      b. Sets this value permanently to thisArg for the new function. This binding cannot be overridden later.
      c. Accepts arguments individually which are pre-set(or curried) in the new function.
      d. Usecase: When needed to create a reusable function with its **this** context permanently locked in, often used in event handlers or when passing methods as callbacks.

```
const person = { name: 'Bob' };
function sayHello(greeting, punctuation) {
  console.log(`${greeting}, my name is
${this.name}${punctuation}`);
}

// 1. Using call()
sayHello.call(person, 'Hi', '!');   // 'this' is
'person'. Args are 'Hi', '!'
// Output: Hi, my name is Bob!

// 2. Using apply()
sayHello.apply(person, ['Hey', '...']); // 'this' is
'person'. Args in an array
// Output: Hey, my name is Bob…

const car = { brand: 'Tesla' };
function getBrand() {
  return this.brand;
}
```

```
// 'this' is NOT set yet. A new function 'boundGetBrand'
is created.
const boundGetBrand = getBrand.bind(car);

// Execute the new function later
console.log(boundGetBrand()); // Output: Tesla
```

Garbage collection

https://medium.com/@raselhasan11/javascript-garbage-collection-a-complete-interview-guide-93e50a2489cd

Event delegation, what is it, how it works, what are pros and cons

Event delegation is a technique in js where a single event listener is attached to a parent element instead of attaching event listeners to multiple child elements. When an event occurs in a child element it automatically bubbles up the dom tree, and the parent's event listener handles it based on the target element.

Pros

1. Improved performance, number of event listeners reduces memory usage and improves overall performance.
2. Dynamic element support: There’s no need to manually attach or remove event listeners when the dom structure changes.

Cons

1. Not all events are bubbled up, there exists non-bubbling events also (focus, blur, scroll, mouseenter, mouseleave, resize etc.
2. Parent component event handler overhead: need to add complex logic to handle each event carefully based on targets.

How to store authentication tokens securely in the browser?

1. Access token should be stored in-memory, so that not accessible by permanent script
2. Refresh token should be stored in secure http only cookie,
    a. Secure, passed over https only
    b. Http-only, not accessible from client side
    c. Same-site, prevents CSRF attacks

Object and prototypes

What is prototype in javascript?

Every javascript object has an internal reference to another object. This is called prototype to that object.

What is an object in javascript?

A collection of key-value pairs, keys are strings or symbols, values can be anything including functions.

What is prototype chain in javascript?

When a property is missing in an object, it looks up in its prototype object, if not found it looks up that object's prototype object and so on. It's called a prototype chain.
const arr = [ ];
Arr -> Array.prototype -> Object.prototype -> null

Difference between __proto__ and prototype?

- ➔ __proto__ exists on all objects and points to object's prototype (**an internal link**)
- ➔ prototype is a property that exists only on constructor functions, and used only if called with a new keyword to create an object. (**property of constructor function**)

What happens when you use **new** keyword?

```
const p = new Person('Rasel');
```

Steps here:

1. Creates an empty object, let say obj.
2. Sets obj.__proto__ = Person.prototype;
3. Call Person with this=obj.
4. Returns the newly created object (obj).

What Object.create() does?

Creates an object with an explicit prototype.

```
const parent = {
      greet() {
            Return 'hello';
      }
          };
const child = Object.create(parent);
child.greet(); // 'hello'
```

Difference between for…in and Object.keys?

```
for (let key in obj) {
```

```
  // iterates own + inherited enumerable properties
}

Object.keys(obj); // only own properties
```

What is deep copy and how can we achieve it?

Creating a full independent clone of an object including all nested objects.

1. **structuredClone**, built-in function, modern javascript way, can handle cyclic references, but functions, class instances handle hoy na
2. **JSON.parse(JSON.stringify(obj))**, older tricky way, doesn't work for date object, undefined and cyclic references.
3. **Lodash cloneDeep** utility function, industry standard, most reliable, preserves almost everything.
4. Manual **recursive deep copy** implementation

```
function deepCopy(obj) {
  if (obj === null || typeof obj !== "object") return obj;

  const result = Array.isArray(obj) ? [] : {};

  for (const key in obj) {
    result[key] = deepCopy(obj[key]);
  }

  return result;
}

const b = deepCopy(a);
```

HTML

Usage of **async** and **defer** attribute in script tag

Async and defer both loads script asynchronously, without blocking page rendering. Defer executes the script only after the dom is fully parsed, and also maintains lexical order of the script's existence in case of script execution.

Difference between iframe and object elements?

Iframe tag is used to embed another html page or external content into a page.
Object tag can embed a variety of content types, such as pdfs and other resources, but is commonly used for embedding entire pages.

What is the <picture> element used for?

To provide multiple image sources for different scenarios. (different screen sizes, types, themes, devices etc.)

```
<picture>
      <source media="(max-width: 600px)" srcset="small-image.jpg">
      <source media="(max-width: 1200px)" srcset="medium-image.jpg">
      <img src="large-image.jpg" alt="Responsive Image">
</picture>
```

What is shadow dom in relation to html elements?

Shadow dom is a browser feature that allows components to have their own isolated dom tree, with encapsulated styles and markup, separated from the main document.

How to load images optimally?

1. Resize kore size choto kora
2. Compress kore size choto kora
3. Webp/avif format use kora
4. Responsive image use kora, srcset e screensize wise different image use kora
5. Lazy loading kora, prioritize kora,
6. Caching, dns use kora

CSS

‘Class’ selector diye multiple elements ke style kora hoy
‘Id’ selector diye unique element style kora hoy, convention holo unique howa lagbe
Pseudo class: user interaction er upor based kore element style korar jonne use kora hoy, like hover, active, nth-child, checked, required, default, valid, invalid, enabled, disabled etc.

Css reset ken use kora hoy?
Browser er default styles overwrite kore custom style apply korar jonne use kora hoy

Pseudo elements ki?
Extra html element use na kore kono html element er aage ba pore kono partial element add kore style korte ::before , ::after use kora hoy

target selections

```
/* target direct children of the parent element */
nav > ul > li {
  color: red;
}

/* target any descendant list items under nav */
```

```
nav li {
  color: blue;
}
```

What is the css box model?
Defines structure of html element using 04 main structural components. (padding, margin, content, border)

Box-sizing ki?
Ekta element er width calculation e padding ar border ke include kora hobe ki hobe na ta box-sizing diye define kora hoy.
Default property hocche content-box, that means width/height excludes border and padding.
Border-box: includes border and padding when calculates width of the html content

Transform ki?
Element ke modify kora, element ke plane er upore move/rotate/scale kora,
onek gula way ache, like

- Translate- obosthan change kora, like daane baame upore niche shorano
- Scale- choto boro kora
- rotate - ghurano

Difference between inline and inline-block display property?
Inline elements height width can't be determined, automatically determined by content, but in inline-block elements, height and width can be changed.

What is flexbox?
Flexbox is a one-dimentional layout model for arranging items, it focuses on alignment, spacing, and distribution.
Use-cases: Component level layouts like navbar, cards row, form controls and centering.

What is grid?
Grid is a two-dimentional layout model for implementing complex layouts.
Use-cases: Full page layout, dashboard, admin panel.

Grid is layout first, flexbox is content first.

Browser APIs

Storage apis

1. localStorage
    a. Data persists permanently until manually cleared, survives page reloads, browser restarts, and tabs.
    b. Capacity; around 5MB
    c. Scope: per origin(protocol+domain+port)
    d. Use-cases:
        i. Saving user preferences (theme, language)
        ii. Caching api responses
        iii. Saving onboarding state, ui layouts, filters.
    e. Limitations:
        i. Not secure for sensitive data
        ii. Synchronous blocking api, heavy operations can affect performance.
2. sessionStorage
    a. Similar to localStorage but lives only for the tab session, cleared when tab closes, each tab gets its own isolated sessionStorage.
    b. Capacity: around 5MB, same as localStorage.
    c. Use-cases:
        i. Data needed only during a session: multi step forms, in-progress / checkout data, temporary ui state
    d. Limitations: Same as localStorage, not secure for sensitive data, synchronous and blocking.
3. Cookies
    a. Small pieces of data sent between browser and server with each request.
    b. Can be session cookies(expire on tab close), or persistent(expire with date).
    c. Capacity: around 4KB
    d. Automatically sent to server with every http request(if not httpOnly strict)
    e. Use-cases:
        i. Authentication sessions(server-managed)
        ii. CSRF tokens
        iii. Tracking (analytics, ads) purposes
    f. Security features:
        i. HttpOnly: client side javascript script cannot access (prevents XSS stealing)
        ii. Secure: Sent over https only
        iii. SameSite: Strict-> no third party cookie access, Lax: Safe GET allowed, None: Cross-site allowed(require secure)

Intersection observer api

Intersection observer api is a browser-optimized, asynchronous api for observing visibility changes of elements relative to a root, commonly used for lazy loading, analytics, infinite scrolling, and performance-critical UI behaviors.

React example:

```
useEffect(() => {
  const observer = new IntersectionObserver(cb, options);
  observer.observe(ref.current);

  return () => observer.disconnect();
}, []);
```

By using intersection observer api pixel-perfect accuracy is not achievable. It's approximate, not frame-perfect. Also not for drag / gesture detection.

Intersection observer shifts visibility detection from javascript to the browser's rendering pipeline.

Why intersection observers faster than scroll listeners?
Because **browser batches intersection checks** and it's **asynchronous**, on the other hand scroll listeners are synchronous, and run outside js call stack timing.

Mutation observer api

Mutation observer api is an asynchronous dom api used to detect structural or attribute changes, mainly useful when dealing with dynamic or third party dom updates outside application control.

Mutation observer api allows us to **react to DOM changes** in a performant asynchronous way without polling. It watches a DOM node and reports structural or attribute changes.

Use-cases:

Integrating with 3rd party scripts
- Ads
- Chat widgets
- CMS-injected html

Detecting dynamic content insertion
- Content loaded outside your control
- SPA route changes without hooks

Auto-initializing components
- Initialize tooltips when elements appear

Why is it safer than polling?
- Async, batched mutations, runs after dom updates.

```
const observer = new MutationObserver((mutations) => {
  mutations.forEach(mutation => {
    console.log(mutation.type);
  });
});

observer.observe(targetNode, {
  childList: true,
  attributes: true,
  subtree: true
});
```

What can be observed?
childList, attributes, characterData(text node changes), subtree(it can be expensive) and mutation record object.

Mutation observer tells me what changed, intersection observer tells me what became visible.

Web vitals

How to improve FCP (First contentful paint)

1. Eliminate render-blocking resources
2. Minify JS/CSS
3. Remove unused js/css
4. Preconnect to required origins
5. Reduce server response times
6. Avoid multiple page redirects
7. Keep request count low and transfer size small

How to improve LCP (Largest contentful paint)

1. Optimize hero image
2. Preload critical resources
3. Minimize render-blocking css/js
4. Use responsive images
5. Improve server response time
6. Avoid lazy loading on LCP element.

Implementation practices:

1. You have a list of buttons. Handle clicks using **event delegation**.

```
<ul id="list">
  <li><button data-id="1">Item 1</button></li>
  <li><button data-id="2">Item 2</button></li>
  <li><button data-id="3">Item 3</button></li>
</ul>
```

Requirements

1. Add one event listener
2. Log the clicked button's data-id

Ans:

```
const listElement = document.getElementById('list');

listElement.addEventListener('click', handleDelegatedClick)

function handleDelegatedClick(e) {
    const button = e.target.closest('button');

    if (!button) return;

    console.log('Button clicked with id:', button.dataset.id);
}
```

2. Implement debounce function

Ans:

```
function debounce(fn, delay) {
    let timeoutId;

    return function (...args) {
        clearTimeout(timeoutId);

        timeoutId = setTimeout(() => {
            fn.apply(this, args);
        }, delay);
    }
}

const debouncedLog = debounce((id) => console.log(id), 2000);
```

3. Implement a deep copy function

Implemented above

4. Implement a throttle function

```
function throttle(fn, delay) {
    let isThrottled = false;

    return function (...args) {
        if (isThrottled) return;

        isThrottled = true;
        fn.apply(this, args);

        setTimeout(() => {
            isThrottled = false;
        }, delay);
    };
}
```

5. Implement example using reduce array method
Input:

```
const users = [
  { name: "Rasel", role: "admin" },
  { name: "Hasan", role: "user" },
  { name: "Ayesha", role: "admin" },
  { name: "Rahim", role: "user" },
  { name: "Karim", role: "guest" }
];
```

Output:

```
{
  admin: ["Rasel", "Ayesha"],
  user: ["Hasan", "Rahim"],
  guest: ["Karim"]
}
```

Ans:

```
const groupedUsers = users.reduce((accumulator, currentUser) => {
    // 1. Destructuring (Optional but good practice)
    const name = currentUser.name;
    const role = currentUser.role;
```

```
    // 2. Conditional Logic to initialize or push
    if(!accumulator[role]) {
        accumulator[role] = [name];
    } else {
        accumulator[role].push(name);
    }

    // 3. Essential: Returning the accumulator for the next loop
    return accumulator;
}, {}); // 4. Essential: The initial empty object {}
```

Top 20 Most Common JS Interview Questions

1. Flatten a nested array

Completely flatten or flatten up to a certain depth
Edge cases: empty arrays, mixed types

Basic:

```
function flatten(arr, depth = 1) {
    const flattenedArr = [];

    for (const item of arr) {
        if (Array.isArray(item) && depth > 0) {
            flattenedArr.push(...flatten(item, depth - 1));
        } else {
            flattenedArr.push(item);
        }
    }

    return flattenedArr;
}
```

Using reduce:

```
function flatten(arr, depth = 1) {
    if (depth <= 0) return arr.slice();

    return arr.reduce((acc, item) => {
        if (Array.isArray(item)) {
            acc.push(...flatten(item, depth - 1));
        } else {
```

```
            acc.push(item);
        }
        return acc;
    }, []);
}
```

2. Deep copy of an object/array

Handle nested objects, arrays, functions, dates

3. Remove duplicates from an array

Using Set, filter, or custom solution

```
function removeDuplicatesDeep(arr) {
    const seen = new Set();

    return arr.filter((item) => {
        const key = JSON.stringify(item);
        if (seen.has(key)) return false;
        seen.add(key);

        return true;
    });
}


function removeDuplicates(arr) {
    return [...new Set(arr)];
}

function removeDuplicates(arr) {
    return arr.filter((item, index) => arr.indexOf(item) === index);
}
```

4. Implement debounce

Limit function calls over time

5. Implement throttle

Limit function calls to fixed intervals

6. First non-repeating character in a string

```
function firstNonRepeatingChar(str) {
  const freq = {};

  // 1st pass: count frequency
  for (const char of str) {
    freq[char] = (freq[char] || 0) + 1;
  }

  // 2nd pass: find first non-repeating
  for (const char of str) {
    if (freq[char] === 1) {
      return char;
    }
  }

  return null; // no non-repeating character
}
```

7. Reverse a string

Iterative, recursive, or using built-ins

```
function reverseString(str) {
  let reversed = "";

  for (let i = str.length - 1; i >= 0; i--) {
    reversed += str[i];
  }

  return reversed;
}

function reverseString(str) {
  return str.split("").reverse().join("");
}

// recursive
function reverseString(str) {
    return recursiveReverseString(str, 0);
}

function recursiveReverseString(str, idx) {
    if (idx === str.length) return '';
```

```
    return recursiveReverseString(str, idx + 1) + str[idx];
}
```

8. Check for palindrome
String or number
Consider ignoring case, spaces, punctuation

```
function checkPalindrome(str) {
    const normalized = str
        .toLowerCase()
        .split('')
        .filter(isValidChar)
        .join('');

    const reversed = normalized.split('').reverse().join('');

    return normalized === reversed;
}
```

9. Longest substring without repeating characters
Sliding window approach
Edge cases: all characters unique or identical

10. Implement flatMap / custom map & flatten
Combine map + flatten logic
Recursion or iteration

```
function flattenMap(arr, mapper) {
    const result = [];

    for (let i = 0; i < arr.length; i++) {
        const mapped = mapper(arr[i], i, arr);

        if (Array.isArray(mapped)) {
            result.push(...mapped); // flatten 1 level
        } else {
            result.push(mapped);
        }
    }

    return result;
}
```

11. Merge two sorted arrays

Iterative merge (like merge sort merge step)
Edge cases: arrays of different lengths

12. Find intersection of two arrays

With/without hash map

Edge cases: duplicates, empty arrays

13. Rotate an array by k positions
Left or right rotation
Edge cases: k > array length, empty array

14. Sum of array elements / max subarray sum
Kadane's algorithm for max subarray sum
Edge cases: all negative numbers

15. Deep equality check between objects

Compare nested objects and arrays

Handle primitive types, arrays, objects

16. Implement once function

Function runs only once

```
function once(fn) {
  let called = false;

  return function (...args) {
    if (called) return;
    called = true;
    return fn.apply(this, args);
  };
}
```

17. Implement curry function
Transform a function into curried form
Variadic arguments or fixed arity

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return function (...nextArgs) {
      return curried.apply(this, args.concat(nextArgs));
```

```
    };
  };
}
```

18. Implement Promise.all

```
Promise.all([p1, p2, p3])
  .then(results => [...])
  .catch(err => ...);
```

Understanding async behavior
Handle resolve, reject, and empty array

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    // edge case
    if (promises.length === 0) {
      resolve([]);
    }

    const results = [];
    let completed = 0;

    promises.forEach((promise, index) => {
      Promise.resolve(promise)
        .then((value) => {
          results[index] = value;
          completed++;

          if (completed === promises.length) {
            resolve(results);
          }
        })
        .catch(reject); // fail fast
    });
  });
}
```

19. Implement cloneDeep manually
Similar to #2, but often asked without JSON methods
Handle circular references

20. Group array of objects by a property
Return an object where keys = property values
Example: group users by age or role

How would you optimize a React application rendering 100k+ items in a list?
What strategies would you use to improve page load time for a global audience?

You notice a memory leak in a production SPA—how do you identify and fix it?

I first reproduce the leak reliably, then use **Chrome DevTools heap snapshots** and allocation timelines to identify growing retained objects like **detached DOM nodes, event listeners, or closures**. I fix the root cause by properly **cleaning up effects, unsubscribing listeners, clearing timers, or reducing closure scope**. Finally, I verify by checking that memory stabilizes after garbage collection and monitor the fix in production.

4️⃣ A component breaks when upgrading a library version—how do you manage dependencies?
5️⃣ How would you debug a performance bottleneck in a React app using DevTools?
6️⃣ You need to migrate a legacy frontend codebase to a modern framework—what's your plan?
7️⃣ How do you ensure secure handling of sensitive user data on the client side?
8️⃣ Users report intermittent UI glitches in different browsers—how would you troubleshoot?
9️⃣ A critical UI feature is failing during peak traffic—how do you mitigate the issue?
🔟 How do you manage state in a complex app to avoid unnecessary re-renders?
1️⃣1️⃣ How would you implement a robust frontend monitoring and logging system?
1️⃣2️⃣ You need to render a large dataset without blocking the main thread—how do you approach it?
1️⃣3️⃣ How would you implement A/B testing without affecting current users?
1️⃣4️⃣ A CSS animation is janky on mobile devices—how do you identify and fix the issue?
1️⃣5️⃣ How do you handle real-time updates in a React application efficiently?

Other reference resources:

Hayder vai: 📕 bKash Questions.pdf